



IMPLEMENTATION REPORT

Admin & Teacher End-to-End Functionality

Every screen, flow, rule and endpoint that ships in the current build — what an administrator can do, what a teacher can do, and exactly how the two meet.

3

Roles shipped

Admin · Teacher · Parent

32

App routes

React Router, role-gated

95

API endpoints

Express, server-authorised

94

Tests passing

7 suites, business rules

Contents

- 1 System overview — stack, architecture, access model
- 2 Shared foundation — sign-in, session, shell, notifications
- 3 Admin: end-to-end functionality
- 4 Teacher: end-to-end functionality
- 5 Where admin and teacher meet
- 6 API surface reference
- 7 Business rules enforced & test coverage
- 8 Data model
- 9 Deferred / not in this build
- 10 Running it — setup, commands, demo accounts

BUILD STATUS

Phase 1 is functionally complete for all three roles. Every AI touchpoint ships as a working manual path with the integration seam already in place; nothing in this document depends on a model. Remaining work is listed in section 9.

SECTION 1

System overview

A homeschool learning portal for Valmiki International Gurukulam, built as three separate packages around one shared contract.

1.1 What the system does

The portal is the single record of a child's learning at VIG. An administrator sets up the curriculum, the teaching staff, the students and the timetable. A teacher turns up to a class, records what happened, and that record fans out into the child's learning map, development evidence and photo history. A parent reads a weekly summary assembled from exactly that material — nothing else, and nothing unapproved.

Three roles, sixteen locked flows. This document covers the administrator and the teacher end to end; the parent surface is described only where the two hand off to it.

1.2 Technology

Tier	Stack	Notes
Frontend	React 19 · TypeScript · Vite · Tailwind v4	PWA, installable, offline shell. React Query for server state.
Backend	Node e 20 · Express · TypeScript	routes ' controllers ' services ' repositories ' Prisma.
Database	Supabase Postgres · Prisma	SQL files are the source of truth; Prisma is generated from them.
Auth	Supabase Auth (server-side only)	Username + password, no OTP. Browser never talks to Supabase.
Storage	Supabase Storage, private buckets	Signed upload and read URLs; media never proxies through the API.
Shared	@vig/shared workspace package	Zod schemas, DTO types, enums — one spelling on both sides of the wire.

1.3 Architecture decisions that shape every flow

Decision	What it means in practice
Express is the only writer	The Supabase data API is disabled and row-level security stays deny-all. Authorisation lives in the service layer next to the business rules, not in policy files.
Server-side sign-in	Supabase Auth is email-keyed, so each user carries a synthetic non-routable alias. The login endpoint resolves username ' alias and signs in on the server; no Supabase token ever reaches the browser.
Media bypasses the API	Uploads go client ' signed URL ' private bucket ' confirm. Reads are short-lived signed URLs issued only after a permission check.
Needs Attention is derived	Operational issues are recomputed from current state on every request rather than stored. An issue disappears the moment its cause is fixed — there is no sync bug to have.

The scheduling engine is pure	It takes an in-memory snapshot and returns ranked options with no I/O, so it is deterministic, unit-testable, and reused for both “find options” and “is this move still valid?”
Route gating is convenience	Client-side role routing exists so people do not see dead ends. Every endpoint re-authorises server-side regardless of what the client rendered.

1.4 Access model

Role alone is never enough. A teacher holds the TEACHER role and still has no business reading another teacher’s class. Every scoped read pushes the constraint into the database query rather than filtering afterwards, so there is no unscoped lookup of student data anywhere in the codebase.

Caller	Students visible	Classes visible	Can write
Admin	All	All	Everything
Teacher	Only students in a class they teach	Only their own occurrences	Class records, learning, development, moments, own availability
Parent	Only linked children	None	Nothing — read-only

ENFORCEMENT

Four guard functions back this table: `assertTeacherOwnsOccurrence`, `assertParentLinkedToStudent`, `assertCanReadStudent` and `readableStudentIds`. List endpoints call the last one and pass the resulting id set into the query itself. These guards are covered by unit tests.

Shared foundation

Everything an administrator and a teacher both touch: getting in, staying in, being told things, and getting out.

2.1 Signing in

Username + password > Server resolves alias > Supabase Auth > Session

- Username and password only — there is no OTP, no email step and no social login.
- The failure message is deliberately generic and never reveals whether a username exists. The API makes the same guarantee, so the two cannot drift.
- Sign-in is rate limited to 30 attempts per 15 minutes per client.
- The response carries the session user: id, username, full name, role, avatar and the mustChangePassword flag.

First login is a gate, not a suggestion

Accounts are created by an administrator with a generated temporary password. Until it is replaced, every API route behind the session guard rejects the call — the client redirect to the change-password screen is only the visible half of it. Auth routes mount ahead of that gate on purpose so a user holding a temporary password can still sign in, read their profile and post the new one.

Forgot password is an operational issue, not an auth channel

There is no email or SMS transport in this deployment. Requesting a reset raises a notification to every administrator, who resets the account, sees the new temporary password once, and passes it on directly. The confirmation shown to the requester is identical whether or not the username exists, and the endpoint is rate limited to five requests per hour.

2.2 The application shell

One shell serves all three roles with a different navigation set. On desktop it is a persistent left rail; on a phone it collapses to a bottom bar and a compact top bar. Content priority, semantic colour and interaction language are identical across both — the layout reflows, it does not miniaturise.

Role	Navigation
Admin	Home · Schedule · Students · Curriculum · Teachers · (Settings)
Teacher	Home · Schedule · Students · Moments · (Settings)
Parent	Home · Learning · Development · Moments · (Settings)

A notification bell sits in both top bars with an unread badge that refreshes every 60 seconds. An offline banner appears when the browser loses connectivity, and an install prompt offers to add the app to the home screen.

2.3 Notifications

Notification volume is a locked product rule rather than a user preference, and it is enforced at the single write path so no future caller can quietly start messaging parents about attendance. A notification whose type is not allowed for the recipient's role is dropped and logged rather than delivered.

Role	May receive
------	-------------

Parent	WEEKLY_UPDATE_READY — exactly one per week, nothing else
Teacher	SCHEDULE_CHANGED, CLASS_RECORD_DUE
Admin	AVAILABILITY_CONFLICT, TEACHER_AVAILABILITY_CHANGE, STUDENT_AVAILABILITY_CHANGE, IN-COMPLETE_SETUP, SCHEDULE_CHANGED, PASSWORD_RESET_REQUEST

The in-app notification centre is the launch channel: a list, a mark-as-read action, mark-all-read, and a tap target that lands on the thing the notification is about. Push subscriptions are collected from day one so the device estate is warm, but delivery stays behind a feature flag until VAPID keys are supplied.

2.4 Settings

Deliberately small: display name, avatar and notification preferences. Role is never user-editable. Push permission is requested and managed from here when the flag is on.

2.5 Progressive web app

- Installable manifest with a generated icon set and an in-app install prompt.
- Service worker: network-first for navigation with a cached shell fallback, cache-first for hashed assets.
- API responses are never cached. Stale school data is worse than no data, and the product forbids implying that editing works offline.

SECTION 3

Admin: end to end

Fourteen screens covering setup, people, curriculum, timetable, publishing and account administration.

3.1 Screen inventory

Home

/admin

Setup checklist while the school is being configured, Needs Attention, today's classes.

Schedule

/admin/schedule

The operational timetable — week grid on desktop, day agenda on mobile.

Add Class

/admin/schedule/new

Structured request ' valid options from the constraint engine ' confirm.

Reschedule

/admin/schedule/reschedule

Propose moves for affected classes, review every old and new time, apply as a set.

Students

/admin/students

Full roster with status, subjects and quick access to each profile.

Add Student

/admin/students/new

Five-step wizard: basics, subjects, availability, parent access, review.

Student Profile

/admin/students/:id

The long-term record plus a separate Manage Student surface for configuration.

Curriculum

/admin/curriculum

Subject ' Level ' Topic ' Skill, one screen per layer.

Teachers

/admin/teachers

Staff list and account creation with a one-time temporary password.

Teacher Profile

/admin/teachers/:id

Capabilities and availability — the two things scheduling actually needs.

Weekly Updates

/admin/weekly-updates

Generate drafts for the week, review, publish to families.

Users

/admin/users

Accounts, status changes and admin-mediated password resets.

Moments

</admin/moments>

The media gallery with educational context.

Class Record

</teacher/class/:id>

Admins may open a teacher's class record flow — the same screen, same rules.

3.2 Admin Home

Home is an operational summary, not a dashboard of statistics. It answers three questions: what is on today, what is blocking it, and — while the school is still being set up — what to do next.

Setup checklist

Setup runs in dependency order and the next incomplete step is the one highlighted, because a scheduling screen with no curriculum is not a screen worth reaching. Completed steps show a green tick and stay reachable; later steps are dimmed. The whole block disappears once all four are done.

1. Curriculum > 2. Teachers > 3. Students > 4. Schedule

Needs Attention

One root cause produces exactly one card, with the affected records hanging off it for drill-down. Three classes broken by one teacher's absence is a single card reading "Priya is unavailable · 3 classes affected", not three unrelated warnings. Cards are ordered danger, then warning, then info. Everything is recomputed on each request over a 14-day look-ahead.

Detected issue	Severity	Grouped by	Action offered
Teacher unavailable for a scheduled class	Danger	Teacher + date	Resolve ' reschedule the affected set
Two classes overlap (same teacher, or a shared student)	Danger	The occurrence pair	Resolve ' open that day
Student scheduled outside their availability	Warning	Student	Review ' student profile
Class records outstanding after the class ended	Warning	Teacher	Review ' open the record
Student setup incomplete (no levels or no availability)	Info	Student	Review student
Password reset requested	Info	User	Reset password ' Users, pre-selected

WHY THIS MATTERS

The reschedule link is not decorative. It carries the exact occurrence ids of the affected classes, so "Resolve" lands on a pre-loaded proposal rather than an empty form the administrator has to rebuild by hand.

3.3 Curriculum

The hierarchy is Subject ' Level ' Topic ' Skill, and each screen drills one layer deeper. Skills are the smallest unit of progress — the thing a student is actually assessed against. Everything above them is structure.



Capability	Available at
Create	Every layer — subject, level, topic, skill
Rename and edit	Every layer
Reorder	Every layer, by explicit ordered id list
Archive	Every layer — never a hard delete, so history stays intact
Read	Teachers read the curriculum too; only admins shape it

Levels carry a display order, and that order is what teacher capability ranges are compared against during scheduling — so curriculum structure feeds directly into who is allowed to teach what.

3.4 Teachers

A teacher's profile answers the two questions the scheduler needs: what can they teach, and when. Both feed straight into schedule validation.

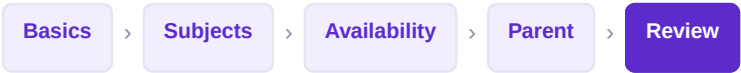
- 1 Create the account
- Username and full name. The system generates a temporary password and creates the linked Supabase auth user, the application user row and the teacher record together.
- 2 Hand over the temporary password
- It is shown once on screen and nowhere else. It is never stored in plaintext and cannot be retrieved again — the administrator passes it on directly.
- 3 Set capabilities
- Subject plus a minimum and maximum level range, with an optional primary flag. This is a hard scheduling constraint, not a note: the engine will not offer a slot for a subject or level the teacher is not marked for.
- 4 Set weekly availability
- A recurring grid of day-of-week time windows.
- 5 Add dated exceptions
- A specific date overrides the recurring pattern on that date only — leave, an extra session, a shifted window.

SELF-SERVICE BOUNDARY

A teacher may read and edit their own teaching profile — availability and exceptions — but capabilities are admin-only, and no teacher can reach another teacher's record at all. The check is a single helper applied at each route.

3.5 Students

Adding a student is more than entering a name. Before a child is teachable and schedulable, VIG needs subjects, a level per subject, weekly availability and parent access. The wizard collects exactly that, then shows a review so mistakes are caught before creation rather than after.



Step	What is captured
Basics	Full name, date of birth, notes, avatar.
Subjects	Which subjects the child studies and the current level in each. This is what the scheduler matches against teacher capability.
Availability	The weekly grid of windows the child can attend. The scheduling engine treats this as a hard constraint.
Parent	Link an existing parent account or create one — this is what grants a family access to the child's record.
Review	Everything above on one screen before anything is written.

The student profile

Viewing and editing are kept apart. The profile reads as a record: identity, current levels, the learning map, development picture, moments and class history. Administrative controls live behind a separate Manage Student surface so the page does not read as a form.

- Subject levels, availability, parent access and status are each edited through their own dedicated endpoint rather than one catch-all save.
- Every change is written to a history trail — what changed, when, and by whom. Nothing is a destructive overwrite.
- Status is ACTIVE, INACTIVE or ARCHIVED; students are never deleted.
- The same list component serves admins and teachers — the API scopes the result, so there is no client-side filtering that could be bypassed.

3.6 Scheduling

This is the most involved part of the administrator's job and the part with the most logic behind it. The timetable itself gives the week at a glance on desktop and a day-oriented agenda on a phone, because a seven-column grid shrunk onto a phone is unreadable.

Creating classes

Structured request > Ranked valid options > Review & confirm > **Recurring classes**

The administrator states what they want — students, subject, level, optional preferred teacher, sessions per week, duration, morning/afternoon/any preference, start date and optional end date. The engine returns only slots that are actually valid, ranked. The administrator picks, reviews, confirms.

THE CONSTRAINT ENGINE

A slot is offered only when all four hold at once:

teacher capability ' teacher availability ' student availability ' no conflict

Dated exceptions override recurring availability on their own date only. The engine is pure and I/O-free — it takes a snapshot and returns ranked options — which is why the same code answers both “find me options” and “is this proposed move still valid?”.

Confirming a slot creates the class and expands it into occurrences: a fixed weekly pattern, respecting the end date and a bounded horizon. Every occurrence is a real row that can be individually cancelled, rescheduled or recorded against.

Rescheduling

When a teacher becomes unavailable, the administrator opens Reschedule — usually straight from a Needs Attention card, which pre-loads the affected occurrences. The system proposes a new time for each. Proposed moves are never applied invisibly: every old and new time is shown, and the whole set is confirmed as one action.

REVALIDATION AT APPLY TIME

The engine re-checks each move again when Apply is pressed. A proposal that has become invalid since it was generated — because something else was booked in the meantime — is dropped rather than forced through. The response reports how many moves actually landed.

Individual occurrences can also be cancelled outright with a reason. Affected teachers receive a SCHEDULE_CHANGED notification; parents do not, because scheduling noise is not something families are messaged about.

3.7 Weekly updates

The weekly update turns a week of approved records into a parent-friendly story. It is not a raw dump of every teacher note, and it is assembled deterministically from template rules — no model is involved.

1 Generate

Per student, or for the whole school at once. The system gathers the week's learning updates, development observations, class notes and moments and builds a draft with a “Week at a glance” narrative.

2 Review

The draft sits in the admin list. Regenerating replaces the item set but keeps the same row, so a published update keeps its identity and a parent's link never breaks.

3 Publish

A deliberate human action. Publishing is the single moment a family hears from us all week, so nothing auto-publishes. On publish the parent receives their one notification, and only then does the update become visible to them.

ONLY APPROVED MATERIAL IS ELIGIBLE

Item selection reads exclusively from saved, teacher-approved records. A parent can never see a draft class record, an unsaved observation, or anything a teacher has not committed.

On the parent side the same update renders as a downloadable PDF, generated in the browser from the same data object the screen renders, so the document and the portal cannot drift apart.

3.8 Moments

The media gallery holds photos and videos with educational context — what happened, when, which students, and which class it came from. Uploading never routes bytes through the API.

Request signed URL

>

Upload direct to bucket

>

Confirm + tag

>

Visible in gallery

- Admins and teachers can upload; a teacher may only tag students they actually teach.
- Filters by student, subject and media type; a per-student gallery is reachable from the profile.
- Reads are short-lived signed URLs issued only after a permission check, so a copied link does not become a permanent public one.

3.9 Users and password resets

Account administration lists every user with role, status and whether they still hold a temporary password. Three actions are available: create, change status, and reset password.

The reset flow is the admin-mediated one described in section 2: a request raises an operational notification, the administrator resets, the new temporary password is displayed once, and the user is forced to replace it on next sign-in.

3.10 AI usage and spend

The AI usage log and its admin endpoint ship now and stay empty until Phase 2. The endpoint reports spend against a configured monthly budget with a soft alert at 80% of it, so the reporting surface is already in place before the first model call is ever made.

3.11 A full administrator journey

How the pieces above compose in practice, from an empty database to a running week.

- 1 Sign in and replace the temporary password**
The account was seeded or created by another admin. Every other route is blocked server-side until the new password is set.
- 2 Home shows the setup checklist**
Four steps in dependency order, with Curriculum highlighted as next.
- 3 Build the curriculum**
Add subjects, then levels within each, then topics, then the skills that will actually be tracked. Ordering matters — level order is what capability ranges are checked against.
- 4 Add teachers**
Create each account, hand over the temporary password, set what they can teach and when they are free. Add any known dated exceptions.
- 5 Add students**
Run the five-step wizard per child. Skipping subjects or availability is possible but Home will immediately surface it as an incomplete-setup issue.
- 6 Create the schedule**
Describe the requirement, review the valid options the engine returns, confirm. Classes expand into weekly occurrences.
- 7 Steady state: work Needs Attention**
Each morning Home shows what is broken — an unavailable teacher, a double booking, an overdue class record, a reset request — and each card links straight to the screen that fixes it.
- 8 End of week: publish**
Generate the drafts for all students, review, publish. Families receive their one notification and their weekly story.

3.12 What the administrator cannot do

- Retrieve an existing password. Temporary passwords are shown once and are never recoverable.
- Publish anything a teacher has not saved — weekly updates only draw on approved records.
- Delete a student, a teacher or a curriculum node. Everything archives instead, so history stays intact.
- Apply a reschedule that has silently become invalid — the engine revalidates and drops it.
- Change a user's role from the settings screen; role is not user-editable anywhere in the UI.

SECTION 4

Teacher: end to end

A deliberately narrow surface. A teacher has one job in this product and the app is shaped around getting it done quickly on a phone.

4.1 Screen inventory

Home

/teacher

The day's work queue: records due, today's classes, what is coming.

Schedule

/teacher/schedule

Their own timetable — the same endpoint the admin uses, scoped server-side.

Students

/teacher/students

Only students in their classes. Same component the admin sees, without the management controls.

Student Profile

/teacher/students/:id

Learning map, development picture, moments, class history — read plus in-scope updates.

Class Record

/teacher/class/:id

The full-screen post-class flow. This is the centre of the teacher product.

Moments

/teacher/moments

Upload and browse media for their own students.

Notifications

/notifications

Scheduling changes and class-record reminders only.

Settings

/settings

Name, avatar, notification preferences.

4.2 Teacher login

Identical mechanics to the administrator: username and password, no OTP, generic failure message, rate limited. The differences are what happens after.

1 First sign-in with the temporary password

The account and the temporary password were created by an administrator and handed over directly. There is no email link to click.

2 Forced password replacement

The change-password screen is a genuine gate. Every API route behind the session guard rejects the call until it is done, not just the client route.

3 Landing on the work queue

After the password is set the teacher lands on /teacher — role-based routing sends each role to its own home, and a teacher attempting an admin URL is redirected back rather than shown an error.

4 Session

Refresh and logout are available; logout forgets the token server-side as well as clearing it client-side.

4.3 Teacher Home

Home is a work queue for the day, not a dashboard. Three sections in priority order, so the next action is obvious without searching a general calendar.

Section	Contents	Tap target
Class records due	Classes that have finished with no saved record yet. Only shown when there are any.	Opens the class record flow
Today	Every class scheduled today, with time, subject, level and student count.	Opens the class record flow
Upcoming	The rest of the week. Falls back to an “all caught up” state.	Read-only

4.4 Schedule and students

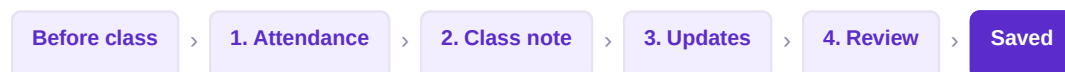
The teacher schedule calls the same endpoint as the administrator timetable. Scoping happens on the server from the caller's teacher id, so there is no client-side filter to get wrong and no way to widen the view by editing a request.

The student list works the same way: an administrator sees every student, a teacher sees only students in a class they teach, and the component itself does no filtering. Attempting to open a student outside that set returns a clear refusal from the API, not an empty page.

4.5 The class record flow

This is where the whole product converges. Attendance, the class note, per-student observations, learning updates, development evidence and media all hang off one class occurrence — there is no second class history anywhere in the system.

The flow runs full-screen with no navigation chrome: a stepper should not compete with the app bar while a teacher is documenting a class on a phone.



Before class

Opening a class shows the roster with avatars, the time and level, and — crucially — the note from the last recorded class with this same group. Continuity without digging through history. If there is none, it says so plainly. One button: Start Class.

Step 1 · Attendance

Present, Absent or Late for every student, as large tap targets. The flow will not continue until every student is marked, because nothing can be recorded about a child who was not there — and the server enforces that same rule independently at save time.

Step 2 · Class note

The overall class note is required: what was covered and how the group responded. Below it, an optional free-text observation per student, and only students marked present are listed.

Step 3 · Optional updates

Everything on this step is optional by design. A single class does not have to generate learning or development data, and the UI says so rather than nudging a teacher into manufacturing progress.

Update type	Fields	Effect once saved
Learning update	Student · skill · new status	Appends to the learning history and moves the skill on the child's learning map. Statuses: To learn, Learning, Needs support, Mastered.
Development observation	Student · area · written observation	Appends evidence under that development area and makes the area appear on the child's profile.

Skill choices are drawn from the level the class is actually teaching, and student choices from the present list only — so an out-of-scope combination cannot be assembled in the first place.

Step 4 · Final review

Everything on one screen before anything is written: attendance chips per student, the class note, each observation, each learning change with its target status, each development observation. Nothing is committed to a child's record until Save is pressed.

Saved

A confirmation screen states exactly what happened and where it went — the note saved to the class history, how many learning updates were applied, how many development observations were added, and that parents will see this in their next weekly update. No silent success.

4.6 What the save actually does

ONE TRANSACTION, ALL OR NOTHING

Attendance, the class note, observations, accepted learning updates, accepted development observations and moment links land together or not at all. A partial save would leave a child's record half-written, which is worse than failing outright.

Rule enforced on the server	Behaviour
The class note is never optional	An empty note is rejected outright, and the note is preserved even when every optional update is skipped.
Absent students receive nothing	Observations and updates for anyone marked absent are filtered out server-side, regardless of what the client sent.
Roster only	Updates for a student who is not in the class are discarded.
A record saves once	Re-submitting an already-saved record is refused rather than duplicating history.
Learning is history plus projection	Each change writes an immutable history row carrying the previous status, then projects the current state — both inside the same transaction.
Development evidence is append-only	Observations are added, never overwritten. The stage a child sits at is a separate, deliberate judgement.

Attendance is written before the record is created so the server already knows who was actually there when it filters what follows. Observations are replaced wholesale on save — the reviewed draft is the teacher's final word.

4.7 Outside the class record

A teacher can also update learning and development directly from a student's profile without waiting for a class — the same endpoints, tagged with a different source so the origin of every change stays visible in the history trail.

- Update a skill status on the learning map for a student they teach.
- Add a development observation, or set a development stage.
- Upload a moment and tag it to their own students.
- Edit their own weekly availability and add dated exceptions — which immediately feed the scheduling engine and can raise a Needs Attention card for the administrator.
- Preview a level change for a student to see which skills remain unfinished before a level is completed.

4.8 What a teacher cannot do

Attempt	Result
Open another teacher's class occurrence	Refused — "This class is not assigned to you."
Open a student they do not teach	Refused — "This student is not in any of your classes."
List all students	The query itself is scoped; the response contains only their own.
Create or reschedule classes	Admin-only endpoints.
Shape the curriculum	Read allowed, write admin-only.
Create accounts or reset passwords	Admin-only.
Generate or publish weekly updates	Admin-only — the entire module requires the admin role.
Set another teacher's capabilities	Admin-only, even for their own record.
Reach an admin URL in the browser	Redirected to their own home; the API would refuse regardless.

4.9 A full teacher journey

- 1 Receive credentials**
Username and a temporary password, handed over by the administrator in person.
- 2 First sign-in**
Replace the temporary password. Nothing else is reachable until this is done.
- 3 Check the work queue**
Home lists anything outstanding from yesterday, then today's classes, then the rest of the week.
- 4 Open the class before it starts**
See the roster and the last note for this group, so the class picks up where it left off.
- 5 Record it afterwards**
Attendance, class note, optional per-student observations, optional learning and development updates, final review, save.
- 6 Add media if there is any**
Photos or videos with context, tagged to the students involved.

7 Keep availability honest

Add an exception for any date they cannot teach. The administrator sees the consequence immediately as a Needs Attention card with a one-click reschedule.

SECTION 5

Where admin and teacher meet

The four handoffs that make the two roles one system rather than two apps.

1 · Account creation 'first sign-in

Admin creates teacher > Temp password shown once > Handed over in person > Teacher replaces it

There is no email transport, so the loop closes through a person. The same path serves a forgotten password: the teacher's request becomes an administrator's Needs Attention card.

2 · Availability 'the schedule

Teacher adds an exception > Engine sees the conflict > Admin Home raises one card > Reschedule set

A teacher marking themselves unavailable does not need to tell anyone. The next time the administrator opens Home, the affected classes are grouped under one card with a link that carries their ids straight into the reschedule proposal.

3 · Class record 'the weekly update

Teacher saves a record > Learning + development written > Admin generates drafts > Publish to family

Nothing reaches a parent by accident. The teacher's save is the approval step, generation only ever reads approved material, and publishing is a separate deliberate human action.

4 · Overdue records 'accountability

A class that has finished without a saved record appears in two places at once: on the teacher's Home under "Class records due", and on the administrator's Home grouped per teacher. Both link to the same record flow. The issue disappears from both the moment it is saved, because attention is derived rather than stored.

5.1 Shared surfaces

Surface	Admin sees	Teacher sees
Student list	Every student, with management controls	Only their own students, read-focused
Student profile	Full record plus Manage Student	Full record, no administrative controls
Schedule	The whole school's week	Their own week — same endpoint, scoped
Class record flow	May open any class record	Only classes assigned to them
Moments	All students	Only their own students
Curriculum	Read and write	Read only, used to pick skills mid-record

ONE IMPLEMENTATION, NOT TWO

These are genuinely the same components and the same endpoints with server-side scoping, not parallel admin and teacher builds. That is what stops the two views drifting apart as the product grows.

SECTION 6

API surface

Every endpoint behind the admin and teacher screens. All paths are prefixed /api/v1. Authorisation is applied server-side on each route.

6.1 Authentication and accounts

	Endpoint	Role	Purpose
POST	/auth/login	Public	Username + password sign-in. Rate limited 30 / 15 min.
POST	/auth/refresh	Public	Exchange a refresh token for a new session.
POST	/auth/logout	Any	Ends the session and forgets the token server-side.
POST	/auth/forgot-password	Public	Raises an admin notification. Rate limited 5 / hour.
GET	/auth/me	Any	The current session user.
POST	/auth/change-password	Any	Replaces a temporary password; clears the gate.
PATCH	/auth/settings	Any	Display name, avatar, notification preferences.
GET	/admin/users	Admin	All accounts with role, status and temp-password flag.
POST	/admin/users	Admin	Creates an account; returns the temporary password once.
PATCH	/admin/users/:id/status	Admin	Active / inactive / archived.
POST	/admin/users/:id/reset-password	Admin	Issues a new temporary password.

6.2 Home and operations

	Endpoint	Role	Purpose
GET	/home/admin	Admin	Setup checklist, Needs Attention, today's classes.
GET	/home/teacher	Teacher/Admin	Records due, today, upcoming.
GET	/attention	Admin	The derived issue list on its own.
GET	/setup/progress	Admin	Which of the four setup steps are complete.
GET	/admin/ai-usage	Admin	Phase-2 spend against budget; empty today.
GET	/health	Public	Database and Supabase status.

6.3 Curriculum

	Endpoint	Role	Purpose
GET	/curriculum/subjects	Any	Subject list, optionally including archived.
GET	/curriculum/subjects/:id	Any	One subject with its levels.

POST	/curriculum/subjects	Admin	Create a subject.
PATCH	/curriculum/subjects/:id	Admin	Rename or edit.
POST	/curriculum/subjects/:id/levels	Admin	Add a level to a subject.
GET	/curriculum/levels/:id	Any	One level with its topics.
GET	/curriculum/levels/:id/skills	Any	Every skill in a level — used by the class record.
PATCH	/curriculum/levels/:id	Admin	Rename or edit.
POST	/curriculum/levels/:id/topics	Admin	Add a topic.
GET	/curriculum/topics/:id	Any	One topic with its skills.
PATCH	/curriculum/topics/:id	Admin	Rename or edit.
POST	/curriculum/topics/:id/skills	Admin	Add a skill.
PATCH	/curriculum/skills/:id	Admin	Rename or edit.
PATCH	/curriculum/:kind/reorder	Admin	Reorder subjects, levels, topics or skills.
POST	/curriculum/:kind/:id/archive	Admin	Archive — never a hard delete.

6.4 Teachers

	Endpoint	Role	Purpose
GET	/teachers	Admin	Staff list.
POST	/teachers	Admin	Create account + teacher record; returns temp password.
GET	/teachers/:id	Admin / self	Profile with capabilities and availability.
PATCH	/teachers/:id	Admin	Name and notes.
PUT	/teachers/:id/capabilities	Admin	Subject + level range. A hard scheduling constraint.
PUT	/teachers/:id/availability	Admin / self	Recurring weekly windows.
POST	/teachers/:id/exceptions	Admin / self	A dated override of the weekly pattern.
DELETE	/teachers/exceptions/:id	Admin/Teacher	Remove an exception.

6.5 Students, learning and development

	Endpoint	Role	Purpose
GET	/students	Any	Scoped in the query: all / taught / linked.
GET	/students/:id	Any	One student, permission-checked.
GET	/students/:id/history	Any	The change trail — what, when, by whom.
POST	/students	Admin	Create from the wizard payload.
PATCH	/students/:id	Admin	Edit core details.
PUT	/students/:id/subject-levels	Admin	Which subjects and the current level in each.
PUT	/students/:id/availability	Admin	Weekly availability grid.
PUT	/students/:id/parent-access	Admin	Link or create the parent account.
PATCH	/students/:id/status	Admin	Active / inactive / archived.
GET	/students/:id/learning	Any	The full learning map.
GET	/students/:id/learning/history	Any	Every skill change over time.
GET	/students/:id/learning/:subjectId	Any	One subject's progress.
PATCH	/students/:id/skills/:skillId	Admin/Teacher	Move a skill status outside a class record.
GET	/students/:id/level-change/preview	Admin/Teacher	Unfinished skills before completing a level.
POST	/students/:id/level-change	Admin	Complete a level and carry forward.
GET	/development/areas	Any	The development area catalogue.
POST	/development/areas	Admin	Add an area.
GET	/students/:id/development	Any	Stages and evidence per area.
GET	/students/:id/development/:areaId	Any	One area in detail.
POST	/students/:id/development/:areaId/observations	Admin/Teacher	Append evidence.
PATCH	/students/:id/development/:areaId/stage	Admin/Teacher	Set the stage — a deliberate judgement.

6.6 Scheduling

	Endpoint	Role	Purpose
GET	/schedule?from&to	Any	Timetable for a range. Teachers are scoped to their own.
POST	/schedule/options	Admin	Ranked valid slots from the constraint engine.
POST	/schedule/confirm	Admin	Create the class and expand recurring occurrences.

POST	/schedule/change/propose	Admin	Propose new times for affected occurrences.
POST	/schedule/change/apply	Admin	Revalidate and apply the whole set.
GET	/classes/:id	Any	One class with its roster.
GET	/occurrences/:id	Any	One occurrence.
POST	/occurrences/:id/cancel	Admin	Cancel with a reason; notifies the teacher.

6.7 Class records

	Endpoint	Role	Purpose
GET	/teacher/occurrences	Teacher/Admin	The teacher's own occurrences.
GET	/occurrences/:id/context	Any	Roster, previous class note, skills, development areas.
GET	/occurrences/:id/attendance	Any	Current attendance.
PUT	/occurrences/:id/attendance	Teacher/Admin	Mark attendance — written before the record.
POST	/occurrences/:id/record	Teacher/Admin	Open (or reopen) the record for this class.
POST	/occurrences/:id/record/draft	Teacher/Admin	Persist an in-progress draft.
GET	/class-records/:id	Any	One record.
PATCH	/class-records/:id	Teacher/Admin	Edit before saving.
POST	/class-records/:id/save	Teacher/Admin	The atomic commit described in 4.6.

6.8 Moments, weekly updates, notifications

	Endpoint	Role	Purpose
GET	/moments	Any	Gallery, scoped and filterable by student, subject, type.
POST	/moments/upload-url	Admin/Teacher	Signed direct-to-bucket upload URL.
POST	/moments	Admin/Teacher	Confirm the upload and tag students.
GET	/moments/:id	Any	One moment, permission-checked.
PATCH	/moments/:id	Admin/Teacher	Edit title, caption or tagged students.
GET	/students/:id/moments	Any	Per-student gallery.
GET	/weekly-updates/current-week	Admin	The current week key and its drafts.

GET	/weekly-updates/students/:id	Admin	Every update for one student.
POST	/weekly-updates/generate	Admin	Build or rebuild one student-week draft.
POST	/weekly-updates/generate-all	Admin	Build drafts for the whole school.
GET	/weekly-updates/:id	Admin	One update with its items.
POST	/weekly-updates/:id/publish	Admin	Publish and notify the family.
GET	/notifications	Any	The recipient's own notifications.
PATCH	/notifications/:id/read	Any	Mark one read.
POST	/notifications/read-all	Any	Mark all read.
GET	/notifications/push/status	Any	Whether push delivery is enabled.
POST	/notifications/push/subscribe	Any	Register a device.
POST	/notifications/push/unsubscribe	Any	Remove a device.

SECTION 7

Business rules and test coverage

The rules that are enforced in code rather than documented in a wiki, and what the suite actually proves.

7.1 Rules enforced in the running system

Rule	Where it lives
The class occurrence is the single anchor for all class data	Class record service — there is no second class history table.
The overall class note is always required and always preserved	Rejected server-side if empty, kept even when every optional update is skipped.
Absent students receive no observations or updates	Filtered server-side at save, from written attendance rather than client claims.
A class record saves atomically or not at all	A single database transaction across six writes.
Learning changes are history plus a projected current state	Immutable update rows carrying the previous status, projected in the same transaction.
Development evidence is appended, never overwritten	Observations only ever insert; stage is a separate action.
Schedule validity = capability ' teacher free ' student free ' no clash	The pure constraint engine, used for both search and revalidation.
Dated exceptions override recurring availability on their date only	Availability resolution, shared by the engine and the attention detector.
One root cause produces exactly one attention issue	Deterministic group keys with affected records nested for drill-down.
Parents see only linked children; teachers only students they teach	Guard functions that push the constraint into the query.
Parents receive exactly one notification per week	A role allow-list at the single notification write path.
Parents never see unapproved material	Weekly update generation reads only saved records.
A temporary password must be replaced before anything else works	A global middleware gate ahead of every non-auth route.
Nothing is hard-deleted	Archive and status transitions throughout.

7.2 Automated tests

CURRENT RESULT

7 test files, 94 tests, all passing. The suite targets the pure business logic everything else is built on — the rules whose failure would be silent and expensive.

Suite	What it proves

Availability resolution	Dated exceptions override recurrence on their date and only that date.
Constraint engine	A slot is offered only when capability, both availabilities and conflict-freedom all hold.
Recurrence expansion	Fixed weekly patterns expand correctly, honouring end dates and the horizon bound.
Needs Attention grouping	One root cause yields one issue, with affected records preserved for drill-down.
Level completion	Unfinished skills are correctly identified for carry-forward.
Permission guards	Parent and teacher scoping is applied inside the query, not after it.
Notification rules	A parent receives exactly one push type; roles cannot receive each other's.

Typechecking runs across all four packages. Integration tests for the atomic class-record save and Playwright end-to-end coverage of the four headline stories are the remaining test work.

Data model

SQL files are the source of truth and are applied in order; the Prisma client is generated from the result rather than the other way round.

File	Contents
001_enums	Every enumerated value — roles, statuses, attendance, skill status, development stage, notification and attention types.
002_identity	Users, teachers, parents and the links between them.
003_curriculum	Subjects, levels, topics, skills.
004_availability	Recurring weekly slots and dated exceptions for teachers and students.
005_learning	Student subject levels, skill progress, and the learning update history.
006_scheduling	Classes, class students, and class occurrences.
007_class_record	Class records, attendance, student observations.
008_development	Development areas, per-student areas, observations and stages.
009_moments	Media items and their student and class links.
010_parent_and_system	Parent-student links, weekly updates and items, notifications, audit log, AI usage log.
011_foreign_keys	Referential integrity applied once the tables exist.
012_rls_deny_all	Row-level security enabled and deny-all as defence in depth.
013_storage_buckets	Private media buckets.
014_reference_data	Seed reference rows such as the development area catalogue.

WHY DENY-ALL RLS

Express is the only writer and the Supabase data API is switched off, so policies would be a second, divergent copy of rules that already live in the service layer beside the business logic. Deny-all means a leaked anon key opens nothing.

An audit trail records administrative changes — who changed what and when — and backs the history views on the student and teacher profiles.

SECTION 9

Deferred / not in this build

Stated plainly, so nothing in this document is read as more finished than it is.

9.1 AI — deferred to Phase 2

No AI is wired anywhere in this build. Every touchpoint ships as a working manual path with the integration seam already in place: the transcription provider, class-note extractor and scheduling interpreter interfaces are defined with manual and no-op implementations. Phase 2 swaps the registration in one container without touching a route or a service.

Touchpoint	What ships today	What Phase 2 adds
Class record capture	Typed fields the teacher fills in.	A voice recorder whose transcript becomes a reviewable draft in the same fields.
Scheduling request	A structured form — already the reviewable, editable request.	A sentence parsed into that same form. The engine behind it is unchanged and ships now.
Reschedule	Select the affected classes explicitly.	A “describe the change” entry point that selects them. Review and apply are unchanged.
Spend reporting	The usage log and admin endpoint, empty.	Real usage against the monthly budget with an 80% soft alert.

WORTH BEING CLEAR ABOUT

The intelligence in scheduling — the constraint engine — is not AI and is not deferred. It ships now, fully tested. What Phase 2 adds there is only the parsing of a sentence into fields the form already collects.

9.2 Other outstanding items

Item	Status
Web push delivery	Subscriptions are collected from day one so the device estate is warm. Delivery sits behind a feature flag — generate a VAPID keypair, set two variables, flip the flag, and wire the transport at the single marked call site.
Integration tests	The atomic class-record save is covered by unit tests of its rules but not yet by a database-level integration test.
End-to-end tests	Playwright coverage of the four headline stories is outstanding.
Institutional branding	The icon set is a placeholder violet mark. The real VIG logo must not be recoloured or redrawn to match the product palette, so the supplied artwork replaces those files when it arrives.
Live credentials	Until Supabase values are supplied the API still starts: health reports degraded, and sign-in and media return a clear 503 rather than crashing.

Running it

Everything needed to stand the system up and sign in as each role.

10.1 Setup

```
npm install
cp .env.example .env      # fill in the Supabase values
npm run setup              # apply schema, sync Prisma, seed demo data
npm run dev                # backend :4000 · frontend :5173
```

Node 20 or newer is required. The setup command is preflight-checked and safe to re-run; use the check variant to validate credentials without changing anything.

10.2 Environment values

Variable	Where it comes from
DATABASE_URL	Supabase ' Settings ' Database ' connection string, pooler, port 6543.
DIRECT_URL	The same page, session pooler, port 5432 — schema changes only.
SUPABASE_URL	Settings ' API ' Project URL.
SUPABASE_ANON_KEY	Settings ' API ' anon public key.
SUPABASE_SERVICE_ROLE_KEY	Settings ' API ' service_role. Server only — never shipped to the browser.

10.3 Commands

Command	What it does
npm run dev	Backend and frontend together.
npm run typecheck	All four packages.
npm test	Business-rule unit tests.
npm run build	Production frontend build.
npm run schema:apply	Apply the SQL schema files in order.
npm run db:pull	Re-sync the Prisma schema and client from the live database.
npm run db:seed / db:studio	Seed demo data · browse the database.
npm run icons	Regenerate the PWA icon set.
npm run grep:ai	List every Phase-2 AI seam left in the codebase.

10.4 Demo accounts

Seeded for evaluation. They share one password and are created without the forced-change flag so the prototype can be explored immediately — real accounts always carry it.

Username	Name	Role	Password
anjali	Anjali Rao	Admin	Valmiki@2026
priya	Priya Sharma	Teacher	Valmiki@2026
meera	Meera Reddy	Teacher	Valmiki@2026
ananya	Ananya Sharma	Parent	Valmiki@2026

SUGGESTED WALKTHROUGH

1. Sign in as anjali — read Needs Attention on Home, open Schedule, open a student profile.
2. Sign in as priya — open a class from Home and run the record flow through to Save.
3. Back as anjali — generate the weekly updates and publish one.
4. Sign in as ananya — read the published update and download it as a PDF.